

Project Profitability Dashboard

Find unprofitable engagements, budget overruns and differences between project revenue, invoices and receipts.

BUSINESS BRIEF | CLIENT REQUIREMENTS | DATA MODEL | STEP-BY-STEP BUILD

What the project does

Find unprofitable engagements, budget overruns and differences between project revenue, invoices and receipts. It turns the specified source tables into an interactive, filterable view of these business outcomes.

Synthetic demo checkpoints

Reference figures below describe the included synthetic dataset only; they are not client results.

REFERENCE KPI	DEMO VALUE
Recognized Revenue	PKR 65,440,000
Delivery Cost	PKR 59,749,800
Project Profit	PKR 5,690,200
Project Margin	8.7%
Budget Remaining	PKR 13,150,200

REQUIREMENTS NOTE | This guide translates the repository specification into an implementation brief. Confirm it with the actual client; it is not a signed discovery or acceptance document.

01 / Business problem and client requirements

Agree the outcome and KPI definitions before loading customer data.

Business problem

Find unprofitable engagements, budget overruns and differences between project revenue, invoices and receipts.

Without consistent definitions, business totals can be difficult to compare or trace back to a responsible team, customer, product, or period. The dashboard is designed to let managers review the specified KPIs, apply business filters, and inspect supporting detail. This is the intended use inferred from the project specification, not a claim about a named client's current process.

Requirements interpreted from the project specification

- Provide a Power BI report that supports the stated goal: Find unprofitable engagements, budget overruns and differences between project revenue, invoices and receipts.
- Show the 3 project pages, their KPI cards, charts, filters, and detail tables listed in the page blueprint.
- Calculate the named measures using their documented formulas and preserve the project rules.
- Use the reporting scope: Jan-Sep 2026 monthly project performance.
- Allow analysis by: Calendar.Month, Projects.Manager, Projects.Service
- Deliver editable PBIP report and semantic-model source, a data contract, private-source templates, and a reconciliation path.

Project boundaries

One row per project/month. Recognized revenue is independent of invoices and cash. CostBudget is allocated monthly, not a repeated full-project budget. Collections can exceed current invoices with opening receivables in real data. Invoice cash gap is not an overdue balance; opening receivables and due dates are outside scope.

Client discovery questions

Confirm source systems and data owners; reporting currency and timezone; period and as-of date; refresh frequency; exact business definitions; privacy and access requirements; who signs off; and whether publishing to Power BI Service is in scope.

02 / Data contract and model

Keep the input tables at their documented grain; the grain determines what a summed KPI means.

TABLE	GRAIN / KEY	FIELDS (SOURCE TYPE)
Projects	one project / ProjectID	ProjectID (string), Project (string), Client (string), Manager (string), Service (string)
Performance	one project per month / PeriodID	PeriodID (string), ProjectID (string), Date (dateTime), RecognizedRevenue (decimal), LaborCost (decimal), SubcontractCost (decimal), Expenses (decimal), ActualHours (int64), PlannedHours (int64), CostBudget (decimal), Invoiced (decimal), Collected (decimal)
Metrics	one metadata row / Label	Label (string)
Calendar	one calendar date / Date	Date (dateTime), Year (int64), Month (string), Quarter (string)

Relationships in the PBIP model

FROM (MANY)	TO (ONE)	FILTERING
Performance.ProjectID	Projects.ProjectID	oneDirection
Performance.Date	Calendar.Date	oneDirection

Data handoff: Use data/import-template with exact filenames and headers. Keep client source files in approved private storage; do not commit them. Set DataMode=Folder and DataFolder to the approved location only after validating the inputs.

03 / KPI and DAX glossary

Treat these formulas and definitions as a signed calculation contract.

MEASURE	DAX SPECIFICATION	MEANING
Recognized Revenue	SUM(Performance[RecognizedRevenue])	Accrual revenue recognized for the reporting period.
Labor Cost	SUM(Performance[LaborCost])	Sum of LaborCost at the recorded grain.
Subcontract Cost	SUM(Performance[SubcontractCost])	Sum of SubcontractCost at the recorded grain.
Expenses	SUM(Performance[Expenses])	Sum of Expenses at the recorded grain.
Delivery Cost	[Labor Cost] + [Subcontract Cost] + [Expenses]	Direct project delivery cost excluding general overhead.
Project Profit	[Recognized Revenue] - [Delivery Cost]	Recognized revenue less direct delivery cost.
Project Margin	DIVIDE([Project Profit], [Recognized Revenue])	Project profit / recognized revenue.
Cost Budget	SUM(Performance[CostBudget])	Monthly allocated direct delivery budget.
Budget Remaining	[Cost Budget] - [Delivery Cost]	Allocated budget less actual cost; positive is under budget.
Actual Hours	SUM(Performance[ActualHours])	Sum of ActualHours at the recorded grain.
Planned Hours	SUM(Performance[PlannedHours])	Sum of PlannedHours at the recorded grain.
Hours Variance	[Actual Hours] - [Planned Hours]	Positive means more hours consumed than planned.
Effective Revenue per Hour	DIVIDE([Recognized Revenue], [Actual Hours])	Recognized revenue per actual delivery hour.
Invoiced	SUM(Performance[Invoiced])	Sum of Invoiced at the recorded grain.
Collected	SUM(Performance[Collected])	Sum of Collected at the recorded grain.
Invoice Cash Gap	[Invoiced] - [Collected]	Selected-period invoices minus collections. Not accounts-receivable balance or overdue debt.

Format currency using the client's approved reporting currency. Show rate, margin, attainment, and share measures as percentages. Format units, hours, days, and counts as whole or decimal numbers only where their source supports it.

04 / Build step by step | 1-5

Work through the stages in order and preserve the original demo as your baseline.

1. Confirm the business decisions

Start with: Find unprofitable engagements, budget overruns and differences between project revenue, invoices and receipts. Confirm who will use the report and what action each page supports. Agree the reporting period, currency, refresh cadence, data owner, KPI definitions, cutoffs, and acceptance owner. The brief here is inferred from this repository spec, not a signed client requirements document.

2. Request the right source extracts

Request the tables at this grain: Projects: one project, key ProjectID; Performance: one project per month, key PeriodID. Use the exact file names, headers, primary keys, and types listed in the data dictionary. Ask for historical periods only where the project scope supports them; do not assume snapshots can reproduce history.

3. Keep data secure and test its quality

Place client files in approved private storage, outside GitHub. Check unique keys, required fields, valid dates, numeric signs, foreign keys, and coverage. Preserve these rules: One row per project/month. Recognized revenue is independent of invoices and cash. CostBudget is allocated monthly, not a repeated full-project budget. Collections can exceed current invoices with opening receivables in real data. Invoice cash gap is not an overdue balance; opening receivables and due dates are outside scope. The demo is synthetic and is for testing the model, not for client reporting.

4. Open the PBIP source project

Download and extract the full repository, then open power-bi/09-project-profitability/Dashboard.pbip in Power BI Desktop. Keep the complete folder structure. PBIP saves report and semantic-model definitions as project files that can be versioned [1].

5. Check the known-good demo baseline

Refresh in Demo mode. Review DATA-CONTRACT.md, validation/expected-demo.json, and Model.SemanticModel/DAXQueries/Reconcile.dax. Record the supplied demo totals and click through every page before replacing the source.

05 / Build step by step | 6-10

Connect private data, build the model, and test the finished report.

6. Connect the client's approved data

Copy the matching CSV templates from `power-bi/09-project-profitability/data/import-template/` to the client's private source folder. In Power Query > Manage Parameters set `DataMode` to `Folder` and `DataFolder` to that absolute path. Keep the required Metrics row. Select Transform Data to inspect types and fix errors before loading [2].

7. Prepare fields and relationships

Use the declared data types, retain one row at the required grain, and handle blanks only as documented. Check these many-to-one links in the model: `Performance.ProjectID > Projects.ProjectID`; `Performance.Date > Calendar.Date`. Lookup tables should filter facts; avoid fact-to-fact joins that multiply values [3].

8. Add and reconcile measures

Use `measures.dax` and the KPI glossary as the calculation contract. Create explicit DAX measures with approved formats. Test results under each date and business-unit filter; verify totals against an independent calculation. Measures respond to the report filter context [4].

9. Build and review the report pages

Recreate the report blueprint below: exact page purpose, slicers, KPI cards, chart series, and detail rows. Add clear titles, sort time labels chronologically, format currency and percentages, and test interactions. Keep project-specific definitions visible to the report user.

10. Sign off, publish, and hand over

Refresh, reconcile source totals, test edge cases, document KPI ownership, and obtain business sign-off. Keep the PBIP source in version control. If service publishing is in scope, publish to the approved workspace, configure credentials and gateway if needed, schedule refresh, and test permissions [5]. Do not publish confidential data before access/security sign-off.

06 / Report blueprint and acceptance

Implement each page to support a specific decision and trace each number to source data.

PAGE / FILTERS	KPI CARDS	CHARTS / DETAIL TABLE
Portfolio profitability Filters: Month, Manager, Service	Recognized Revenue, Project Profit, Project Margin, Budget Remaining	Charts: Project profit by service; Recognized revenue and delivery cost by month Detail: Project, Recognized Revenue, Delivery Cost, Project Profit, Project Margin
Delivery and budgets Filters: Month, Manager, Service	Delivery Cost, Cost Budget, Hours Variance, Effective Revenue per Hour	Charts: Delivery cost vs budget by manager; Actual vs planned hours by service Detail: Project, Cost Budget, Delivery Cost, Budget Remaining, Hours Variance
Billing and collections Filters: Month, Manager, Service	Invoiced, Collected, Invoice Cash Gap, Recognized Revenue	Charts: Invoices and collections by client; Invoices and collections by month Detail: Project, Invoiced, Collected, Invoice Cash Gap

Acceptance checklist

- Open the PBIP in current Power BI Desktop and refresh all tables without errors.
- Reconcile source counts, key uniqueness, relationships, and every project-specific data rule.
- Compare KPI measures to independently calculated totals and the supplied expected-demo.json.
- Test each slicer, filter interaction, date behavior, chart title, table detail, blank context, and zero denominator.
- Confirm currency, percentage formats, report audiences, privacy, access, refresh ownership, and client sign-off.

Demo acceptance values

METRIC	EXPECTED (SYNTHETIC)
Recognized Revenue	PKR 65,440,000
Delivery Cost	PKR 59,749,800
Project Profit	PKR 5,690,200
Project Margin	8.7%
Budget Remaining	PKR 13,150,200
Invoice Cash Gap	PKR 12,686,434.07

07 / Handover and references

Use the Microsoft guidance below for product workflows; verify tenant-specific settings at delivery.

- Document approved data owner, source, refresh schedule, and support contact.
- Keep the metric dictionary, data contract, sign-off totals, and known limits with the delivered report.
- For Service deployment, confirm target workspace, credentials, gateway needs, permissions, tenant policies, and applicable license.
- A slicer does not enforce row-level security. Design and test security separately before exposing restricted data.
- This repository contains synthetic examples and a source implementation. Desktop rendering, engine evaluation, Service deployment, and client acceptance are separate release gates.

Microsoft Learn references

[1] Power BI Desktop projects (PBIP) - project format and folders

<https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview>

[2] Connect to data in Power BI Desktop - Get Data and Power Query

<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-connect-to-data>

[3] Understand star schema and the importance for Power BI

<https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>

[4] Tutorial: Create your own measures in Power BI Desktop

<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-tutorial-create-measures>

[5] Publish semantic models and reports from Power BI Desktop

<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-upload-desktop-files>

References checked 30 September 2026. Power BI interface labels, preview features, licensing, and tenant controls can change; confirm them in the client's environment.